Burger Ordering Website – Full System Description (Updated with Customization, Allergies, and Resident Details)

The Burger Ordering Website is a comprehensive food ordering system designed to efficiently manage burger sales, customer orders, stock levels, business operations, and customer notifications within a residential building. It is built for scalability, reliability, and high availability, ensuring smooth operations even during high-demand periods.

Frontend / Customer Interface

The customer-facing website provides a user-friendly experience:

Product Browsing

• Customers see only burgers, sides, drinks, and sauces that are available and in stock.
• Out-of-stock or disabled items are automatically hidden.
• Lazy loading of images ensures faster page load times.
• Images are served from Supabase Storage or CDN for optimized delivery.

Ordering & Customization

• Customers select a base product (e.g., burger).
• **Extras** (cheese, bacon, etc.) can be added; defined by admin with prices.
• **Sides** (fries, onion rings, etc.) can be added; defined by admin with prices and images.
• **Special Instructions / Allergies**: Customers can write notes such as allergies, preferences, or anything the kitchen should know.
• Orders below a minimum threshold are blocked to maintain business viability.
• Supports group orders with split payment.

Resident Delivery Info

• Building address is the same for all residents.
• Customers must provide:

- **Block** (B or C) → dropdown selection
- **Room number** → input field
  • Delivery uses **block + room number** to locate customers accurately.

Order Review

• Customers can review the full order with:

- Base product

- Selected extras and their prices
- Selected sides with images and prices
- Special instructions / allergies
- Block + room number
  • Total price is dynamically calculated.
  • Customers can modify or remove extras/sides before checkout.

Real-Time Updates

• Customers receive notifications for:

- Order placement
- Order status changes (Processing → On The Way → Delivered)
- Business open/close status
  • Notifications are sent via in-app, SMS, WhatsApp, or email (multi-provider system).

Customer Accounts

• Registration to save payment methods and delivery details (block + room number).
• Quick reorder buttons for frequent customers.
• Order history tracking for convenience.

Ratings & Feedback

• Customers can rate orders and delivery experience.
• Feedback informs business improvements.

Admin Dashboard

The admin side provides full control over operations:

Product Management

• Add, edit, or remove burgers, sides, drinks, and sauces.
• Set prices and manage availability.

Stock Management

• Track quantity of each item.
• Automatic stock updates when orders are placed.
• Atomic stock updates prevent overselling during high concurrency.

Business Hours Control

• Open or close the business with a single button.
• Customers receive notifications when the business status changes.

Order Management

• Track every order from placement to delivery.
• Update order status: Processing → On The Way → Delivered.
• Queue-based order handling ensures no overload during high traffic.

Admin Improvements

• Bulk operations for editing multiple products or prices.
• Export functionality (Excel/CSV) for reporting.
• Audit logs to track changes made by admins.

Backend System & Workflow

Order Processing Workflow

1. Customer submits order with:
    - Selected product
    - Selected extras
    - Selected sides
    - Special instructions / allergies
    - Block and room number
2. System validates:
    - Business is open
    - Product, extras, and sides availability
    - Stock quantity
    - Block and room number are provided
3. Order is queued for processing (BullJS / RabbitMQ).
4. Worker processes orders asynchronously:

- Reduces stock quantities
- Saves order in database with instructions and resident info
- Sends notifications and emails

Order Status Management

• Admin updates status through structured workflow:
Processing → On The Way → Delivered
• Customers receive real-time updates.

Database & Storage

Database Structure

Products (base items)

id | name | price | image_url | availability

Extras

id | name | price | availability

Sides

id | name | price | image_url | availability

Customers

id | name | block | room_number | phone | email | payment_methods

Orders

id | customer_id | total_price | status | special_instructions | created_at

Order_Items

id | order_id | product_id | quantity | total_price

Order_Item_Extras

id | order_item_id | extra_id | quantity

Order_Item_Sides

id | order_item_id | side_id | quantity

Notifications

id | order_id | type | status | timestamp

Storage

• Images (burgers, sides, drinks) are stored in Supabase Storage.
• Only URLs are saved in the database.
• Optionally, a CDN serves images to improve performance.

Scalability & Performance

Connection Management

• Connection pooling ensures no more than 50 simultaneous connections to Supabase.
• Orders exceeding connection limits are queued.
• Batch inserts reduce the number of database transactions.

Caching

• Frequently read data (product catalog, stock levels, notifications) cached using Redis or Supabase Edge Cache.

Read Replicas

• Real-time updates go to the primary database.
• Analytics queries run on read replicas to avoid load on the primary DB.

High Availability

• Replication prevents downtime.
• Daily automated backups for disaster recovery.
• Optionally, images and backups can be stored externally (e.g., S3).

Notifications & Multi-Email Provider Strategy

Multi-Provider Email Strategy

Email Type	Provider	Purpose
Welcome emails	Provider A	Sent on customer registration
Order confirmations & status updates	Provider B	Sent for orders

Email Type	Provider	Purpose
Business open notifications	Provider C	Sent when business opens

Queue-Based Email Sending

• Emails are queued and sent asynchronously via a backend worker.
• Failures are retried automatically.
• Logs of sent/failed emails are maintained in the database.

Analytics, Scheduling & Reporting

• Sales dashboards with visual charts for peak hours, revenue trends, and popular items.
• Customer insights track ordering patterns and preferences.
• Peak hour preparation scheduling allows advance preparation of popular items.
• Background jobs handle tasks like email/SMS notifications and report generation asynchronously.

Frontend Optimization

• Lazy loading of images reduces initial load time.
• CDN used for static assets (CSS, JS, images) to reduce server load.
• Supabase Storage URLs are efficiently utilized for fast image retrieval.

High Availability & Reliability

• Database replication ensures minimal downtime.
• Queue-based order and email handling prevent crashes during high load.
• Offline mode allows basic ordering in case of internet issues.
• Disaster recovery plan includes backups of database and storage assets.

Summary

The Burger Ordering Website is a robust, scalable, and professional food ordering system that combines:

• Customer-friendly frontend for browsing, customizing, and reviewing orders
• Admin dashboard for full operational control

- Queue-based backend for safe order and email processing
- Scalable database design with caching, partitioning, and replication
- Multi-email provider strategy to stay within free-tier limits
- Optimized storage and delivery of images
- Accurate delivery using **block + room number** for residents
- High availability and disaster recovery

This design ensures the system can handle high concurrency, large order volumes, and real-time notifications efficiently while maintaining performance, reliability, and a professional user experience.